



Universidad Católica del Uruguay
Facultad de Ingeniería y Tecnología
Departamento de Ciencias Exactas y Naturales

Cálculo Aplicado

Error Numérico

Errores de redondeo en sumatorias con números de
punto flotante

Emilio Rodriguez
Micaela Crocco
Nicolás Márquez
Gonzalo Juarez

Docentes: Maglis Mujica y Cándido De Oliveira

Octubre 2026

Resumen

En este trabajo se estudian los errores numéricos que aparecen al implementar sumatorias en una computadora utilizando aritmética de punto flotante de doble precisión (IEEE 754). A partir de tres experimentos implementados en Python se analiza si propiedades que se cumplen en la aritmética exacta se conservan en la práctica. En primer lugar, se evalúa la propiedad asociativa mediante la sucesión $a_N = \sum_{k=1}^N (1 + b^k - b^k)$: con enteros el resultado es exacto, mientras que con `float` la suma se estanca por absorción cuando b^k supera aproximadamente 2^{53} . En segundo lugar, se estudia el efecto del orden de los términos en $b_N = \sum_{k=1}^N \frac{1}{k(k+1)}$ comparando cuatro algoritmos de suma: la suma de mayor a menor módulo acumula los mayores errores, mientras que la suma de menor a mayor módulo y la suma compensada de Kahan mantienen el error relativo cerca de la precisión de máquina. Por último, se comparan representaciones matemáticamente equivalentes de b_N y de $c_N = \sum_{k=1}^N (\sqrt{k^2 + 1} - k)$. Para c_N , la forma que resta raíces sufre cancelación catastrófica: su error relativo es entre 3 y 4 órdenes de magnitud mayor que el de la forma racionalizada y, analizando cada término por separado, la resta pierde toda la precisión a partir de $k \approx 6,7 \cdot 10^7$. Los resultados muestran que la elección del algoritmo y de la forma de escribir una expresión es determinante para la precisión de un cálculo numérico.

Índice

1. Introducción	3
1.1. Objetivos	3
2. Marco Teórico	4
2.1. Sumatorias	4
2.2. Representación numérica en computadoras	4
2.3. Propiedad asociativa	5
2.4. Propiedad conmutativa y orden de los términos	5
2.5. Algoritmos de suma y precisión numérica	6
2.6. Error numérico	6
2.7. Representaciones matemáticamente equivalentes	6
2.8. Cancelación catastrófica	7
3. Metodología	7
3.1. Propiedad asociativa	7
3.2. Análisis de impacto del orden de las sumatorias	8
3.3. Representaciones matemáticamente equivalentes	8
4. Resultados	10
4.1. Propiedad asociativa	10
4.1.1. Bonus de propiedad asociativa	10
4.2. Propiedad conmutativa	15
4.3. Representaciones de b_N	17
4.4. Comparación de las formas cerradas	18
4.5. Representaciones de c_N	19
4.6. Términos individuales de c_N	20
5. Discusión	21
5.1. Análisis de la asociatividad	21
5.2. Análisis del orden de los términos	23
5.3. Análisis de las representaciones de b_N	23
5.4. Análisis de las formas cerradas	24
5.5. Análisis de las representaciones de c_N	24
5.6. Comparación de errores / comportamiento numérico	25
6. Conclusiones	26
7. Bibliografía	27

1. Introducción

Las sumatorias son una de las herramientas más utilizadas en matemática y en computación: aparecen en el cálculo de promedios, en el análisis estadístico, en la probabilidad y en la aproximación numérica de funciones, entre otros contextos. En la teoría, la suma es una operación exacta, conmutativa y asociativa, y dos expresiones algebraicamente equivalentes representan siempre el mismo valor.

Al implementar estas operaciones en una computadora la situación cambia. Los números reales se representan con una cantidad finita de bits, habitualmente siguiendo el estándar IEEE 754 de doble precisión (IEEE Computer Society, 2019), por lo que solo un subconjunto finito de valores puede almacenarse de forma exacta y cada operación puede introducir un error de redondeo (Goldberg, 1991). Aunque cada uno de estos errores es muy pequeño, al acumularse a lo largo de miles o millones de operaciones, o al combinarse con operaciones desfavorables como la resta de números muy próximos, pueden producir resultados que se alejan del valor teórico de manera apreciable (Higham, 2002).

Este problema es especialmente relevante en informática e ingeniería, donde una mala elección del algoritmo o de la forma de escribir una expresión puede generar resultados incorrectos sin que el programa emita ningún aviso. Por esta razón, en el presente trabajo se utilizan distintas sumatorias, cuyo valor exacto es conocido, como herramienta para observar, medir y explicar algunos de los fenómenos clásicos del error numérico.

1.1. Objetivos

El objetivo general del trabajo es analizar experimentalmente cómo la representación finita de los números en una computadora afecta el resultado de sumatorias que, en aritmética exacta, tienen un valor conocido. Para ello se plantean los siguientes objetivos específicos:

- Estudiar si la propiedad asociativa se conserva al implementar $a_N = \sum_{k=1}^N (1 + b^k - b^k)$ con distintas agrupaciones de los términos, comparando el uso de valores de tipo `int` y `float`.
- Analizar cómo el orden en que se suman los términos de b_N influye en el error relativo, comparando la suma de mayor a menor módulo, de menor a mayor módulo, la suma randomizada y la suma de Kahan.
- Comparar representaciones matemáticamente equivalentes de b_N y c_N , cuantificando las diferencias numéricas entre ellas.
- Identificar el fenómeno de cancelación catastrófica en el cálculo de c_N y determinar a partir de qué valores de k la pérdida de precisión se vuelve apreciable.

El documento se organiza de la siguiente manera: en el Marco Teórico se presentan los conceptos necesarios para interpretar los experimentos; en la Metodología se describe cómo fueron implementados; en Resultados se presentan los datos y gráficos obtenidos; en la Discusión se analizan y explican esos resultados; y finalmente se exponen las conclusiones generales del trabajo.

2. Marco Teórico

2.1. Sumatorias

La sumatoria es una manera de representar una serie de sumas. Tiene un límite inferior, que es donde comienza la sumatoria, y un límite superior, que es donde termina la misma. En ella, una variable k toma el valor del límite inferior y va incrementándose de un término a otro hasta llegar al límite superior. Además, está acompañada por una fórmula explícita a la derecha de la letra sigma, la cual es una letra griega mayúscula (OpenStax, 2022).

Por ejemplo, la siguiente expresión representa una sumatoria:

$$\sum_{k=1}^5 k$$

En esta expresión se pueden identificar los siguientes elementos:

- k : índice de la sumatoria.
- 1: límite inferior.
- 5: límite superior.
- k : fórmula o término que se va sumando.

Por lo tanto, la expresión anterior equivale a:

$$\sum_{k=1}^5 k = 1 + 2 + 3 + 4 + 5 = 15$$

2.2. Representación numérica en computadoras

Los números de punto flotante son representados en el hardware de las computadoras en base 2, es decir, de manera binaria. El estándar que define esta representación es el IEEE 754, utilizado por la gran mayoría de los dispositivos electrónicos y lenguajes de programación actuales. Este estándar establece, para el caso de doble precisión (`double`), un límite de 53 bits para representar la mantisa (la parte significativa del número).

El problema surge porque, así como en base 10 existen fracciones que no tienen una representación finita (por ejemplo, $1/3 = 0,3333\dots$), en base 2 ocurre lo mismo con muchas fracciones decimales que en base 10 sí son exactas. Un caso típico es el número 0,1, que en base decimal se escribe con una sola cifra, pero en binario es periódico:

$$0,1_{10} = 0,0001100110011\dots_2$$

Como la computadora solo dispone de 53 bits para la mantisa, esta expresión binaria infinita debe redondearse, lo cual introduce un error de representación. Este error, aunque diminuto, puede manifestarse en los resultados de operaciones aritméticas simples. Por ejemplo, al ejecutar en Python:

```
>>> 0.1 + 0.2
0.30000000000000004
```

El resultado no es exactamente 0,3, sino que arrastra el error de redondeo propio de la representación binaria de 0,1 y 0,2. Esto evidencia que el error numérico no es un defecto del lenguaje de programación, sino una consecuencia directa de cómo el hardware almacena los números reales (Python Software Foundation, s. f.-b).

2.3. Propiedad asociativa

La propiedad asociativa es la propiedad que establece que al sumar o multiplicar tres términos o mas, se puede modificar de diferentes maneras como los términos se agrupan sin afectar el resultado final.

Para la suma, esta propiedad puede expresarse como:

$$(a + b) + c = a + (b + c)$$

Por ejemplo:

$$(2 + 3) + 4 = 2 + (3 + 4) = 9$$

2.4. Propiedad conmutativa y orden de los términos

En matemáticas, la adición es una operación estrictamente conmutativa y asociativa. Sin embargo, en la aritmética de punto flotante, el orden en el que se suman los términos afecta el resultado final debido a los errores de truncamiento y redondeo. Cuando se suma una gran cantidad de términos de magnitudes muy dispares, el acumulador principal crece. Si a un acumulador grande se le suma un número muy pequeño, los dígitos menos significativos de este último se pierden para alinear los exponentes, introduciendo un error que se acumula iteración tras iteración.

2.5. Algoritmos de suma y precisión numérica

Para mitigar la pérdida de precisión en sumatorias extensas, el orden de los sumandos y la estrategia algorítmica son determinantes. Existen distintos enfoques:

- **Suma ingenua (Mayor a menor / Menor a mayor):** Acumula secuencialmente. Sumar desde el valor más pequeño al más grande permite que los términos diminutos se acumulen entre sí antes de sumarse a valores mayores, preservando su precisión temporalmente.
- **Suma de Kahan:** Es un algoritmo de suma compensada. Utiliza una variable auxiliar para capturar los pequeños errores de truncamiento que ocurren en cada paso aritmético y los re introduce en la siguiente iteración, logrando resultados sumamente precisos incluso para secuencias largas (Kahan, 1965).

2.6. Error numérico

El error numérico es la diferencia entre un valor aproximado obtenido mediante un cálculo y el valor exacto o de referencia. Este error puede surgir, entre otras causas, por la representación finita de los números y por el redondeo realizado. Una manera de medir todo eso es a través del error absoluto y el error relativo. El error absoluto es la magnitud de la diferencia entre el valor aproximado y el valor exacto:

$$E_{\text{abs}} = |x_{\text{aprox}} - x_{\text{exacto}}|$$

El error relativo relaciona ese error con el valor exacto:

$$E_{\text{rel}} = \frac{|x_{\text{aprox}} - x_{\text{exacto}}|}{|x_{\text{exacto}}|}$$

2.7. Representaciones matemáticamente equivalentes

Dos expresiones pueden ser matemáticamente equivalentes cuando representan el mismo valor según las reglas de la aritmética. Sin embargo, esta equivalencia no garantiza que ambas expresiones produzcan el mismo resultado cuando son evaluadas mediante una computadora. En la aritmética de punto flotante, los números se representan utilizando una cantidad finita de cifras, por lo que algunas operaciones requieren redondeo. Por esta razón, la forma en que se escribe o se reorganiza una expresión puede alterar el resultado numérico.

2.8. Cancelación catastrófica

La cancelación catastrófica es un fenómeno que puede aparecer en cálculos realizados con aritmética de punto flotante cuando se restan dos valores muy similares. Al realizar esta operación, los dígitos significativos que ambas cantidades tienen en común se cancelan. Si los valores originales contienen errores de redondeo, estos errores pueden alterar de manera importante el resultado final. Esto no implica necesariamente que la resta introduzca por sí misma el error. En cambio, la operación de resta puede poner en evidencia errores que ya habían sido introducidos durante operaciones anteriores, como multiplicaciones o evaluaciones de funciones en punto flotante (Goldberg, 1991).

3. Metodología

Para el desarrollo de este trabajo, específicamente en la estructuración de ideas y el diseño del código, se consultó el modelo de lenguaje Claude (Sonnet 5.5), desarrollado por Anthropic. Todas sus sugerencias fueron revisadas, probadas y adaptadas por los autores. Todos los experimentos se implementaron en Python 3, utilizando la biblioteca `NumPy` para el procesamiento de los resultados y `Matplotlib` para la generación de los gráficos. En ninguna de las implementaciones se utilizó la función `sum` de Python. El código fuente completo, junto con las figuras generadas y las instrucciones para reproducir los experimentos, se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/micaelacrocco/calculo-aplicado-sumatorias>

Cada experimento corresponde a un módulo del repositorio (`sec2_asociativa.py`, `sec3_commutativa.py` y `sec4_representaciones.py`), y la totalidad de los experimentos puede ejecutarse mediante el comando `python main.py`.

3.1. Propiedad asociativa

Para analizar el efecto de la representación numérica en las computadoras y de la forma de agrupar las operaciones, se implementaron diferentes expresiones matemáticamente equivalentes en Python. La idea fue comparar el comportamiento de las expresiones cuando se utilizan números enteros y números de punto flotante.

Se implementaron dos funciones principales. La primera, `suma_con_enteros`, la cual utiliza valores de tipo entero (`int`), mientras que la segunda, `suma_con_flotantes`, utiliza valores de tipo punto flotante (`float`). Ambas funciones calculan tres expresiones diferentes:

$$\begin{aligned} 1 + b^k - b^k \\ (1 + b^k) - b^k \\ (1 - b^k) + b^k \end{aligned}$$

Estas expresiones son matemáticamente equivalentes, ya que en los tres casos el término que depende de b^k se cancela y el resultado esperado para cada término es 1.

Para cada expresión se acumularon los resultados, utilizando un índice k que comienza en 0 y continúa hasta $N - 1$. En cada iteración se almacenó el resultado acumulado, para posteriormente hacer una gráfica con los resultados.

Para los experimentos se utilizaron los valores:

$$b \in \{2, 3, 5, 10\}$$

y se estableció $N = 1,000$. De esta manera, se obtuvieron tres curvas para cada valor de b , correspondientes a las tres formas de realizar las operaciones.

El experimento se realizó en dos situaciones. En la primera se utilizaron números enteros, mientras que en la segunda se utilizaron números de punto flotante. Las gráficas fueron generadas mediante la biblioteca `Matplotlib` y luego se exportaron las gráficas a imagen, para analizar los resultados.

Como extensión (Bonus), el experimento se repitió con `b1` y con valores negativos de b , tanto de tipo `int` como `float`.

3.2. Análisis de impacto del orden de las sumatorias

Para el análisis del impacto del orden en las sumatorias, se utilizó la sucesión $b_N = \sum_{k=1}^N \frac{1}{k(k+1)} = \frac{N}{N+1}$. Se implementaron cuatro algoritmos en Python sin utilizar funciones nativas: suma de mayor a menor módulo, suma de menor a mayor, suma randomizada (mezclando los términos aleatoriamente) y la suma compensada de Kahan. Se calculó el error relativo respecto a la forma cerrada teórica en dos escalas. Para el rango pequeño (de $N = 10$ a $10,000$), se evaluó el error en incrementos de 10. Para el rango grande (de $N = 1,000$ a $1,000,000$), se utilizó un salto de 5,000 términos; esta decisión se tomó para optimizar el tiempo de ejecución computacional, dado que recalculer iterativamente series de tal magnitud tiene un costo excesivo y la tendencia asintótica del error acumulado se puede apreciar con total claridad utilizando esta resolución. Adicionalmente, para comprobar empíricamente la variabilidad del error por falta de asociatividad estricta, se ejecutaron 50 repeticiones de la suma randomizada para un N fijo de 500,000.

3.3. Representaciones matemáticamente equivalentes

Para estudiar si distintas representaciones matemáticamente equivalentes producen los mismos resultados en una computadora, se implementaron dos formas de calcular la sucesión

$$b_N = \sum_{k=1}^N \frac{1}{k(k+1)}.$$

La primera forma calcula directamente cada término como $\frac{1}{k(k+1)}$, mientras que la segunda utiliza la representación telescópica

$$b_N = \sum_{k=1}^N \left(\frac{1}{k} - \frac{1}{k+1} \right).$$

Ambas formas fueron implementadas en Python sin utilizar la función `sum`. Para cada valor de N , se calculó el error relativo de cada representación respecto de la forma cerrada

$$b_N = \frac{N}{N+1}.$$

Los valores de N utilizados fueron $N = 1$ y luego valores desde 10 hasta 10,000, con incrementos de 10. Los errores obtenidos se representaron gráficamente utilizando una escala logarítmica en el eje vertical.

Además como requiere el bonus, se compararon las dos expresiones cerradas matemáticamente equivalentes

$$1 - \frac{1}{N+1}$$

y

$$\frac{N}{N+1}.$$

En este caso, los errores se calcularon respecto del valor exacto $\frac{N}{N+1}$ representado mediante la clase `Fraction`, evitando introducir un error adicional en el valor de referencia. También se comparó directamente el resultado de ambas expresiones para valores de N entre 1 y 1,000,000.

De forma análoga, se implementaron las dos representaciones de la sucesión

$$c_N = \sum_{k=1}^N \frac{1}{\sqrt{k^2+1} + k} = \sum_{k=1}^N \left(\sqrt{k^2+1} - k \right),$$

a las que se denomina forma racionalizada ($c_N^{(1)}$) y resta de raíces ($c_N^{(2)}$), respectivamente. Como c_N no tiene una fórmula cerrada, se construyó un valor de referencia utilizando el módulo `decimal` de Python con 50 dígitos significativos (Python Software Foundation, s.f.-a), una precisión muy superior a la de un `float`. Para los mismos valores de N utilizados con b_N se calculó el error relativo de cada representación respecto de esta referencia y la diferencia absoluta entre ambas:

$$D_N = \left| c_N^{(1)} - c_N^{(2)} \right|.$$

Por último, para el segundo bonus se compararon individualmente los términos

$$\sqrt{k^2+1} - k \quad \text{y} \quad \frac{1}{\sqrt{k^2+1} + k}$$

para 400 valores de k distribuidos de forma logarítmica entre 1 y 10^{10} , calculando el error relativo de cada expresión respecto del término calculado con 50 dígitos. Para cuantificar a partir de qué valor de k la pérdida de precisión es apreciable, se registró el primer k en el que el error relativo de la resta supera distintos umbrales.

4. Resultados

4.1. Propiedad asociativa

En primer lugar, se evaluaron las tres expresiones utilizando valores enteros. Como se observa en la Figura 1, las tres formas de realizar las operaciones producen el mismo comportamiento para todos los valores de b analizados. En los tres casos, el resultado acumulado aumenta linealmente con k , alcanzando un valor de 1,000 al finalizar.

Al utilizar números de punto flotante se observa un comportamiento diferente. Como se muestra en la Figura 2, las curvas dejan de crecer luego de llegar a un determinado valor. Además, el punto en el que esto ocurre depende del valor de b utilizado.

Para $b = 2,0$, el resultado acumulado se estabiliza aproximadamente en 53, mientras que para $b = 3,0$, $b = 5,0$ y $b = 10,0$ se estabiliza aproximadamente en 36, 23 y 16, respectivamente. Este comportamiento es muy parecido en las tres expresiones evaluadas.

4.1.1. Bonus de propiedad asociativa

Como extensión del experimento principal, se analizaron los casos adicionales propuestos en la consigna, considerando valores de b menores o iguales que 1, valores negativos y el comportamiento de los enteros cuando se utilizan números de gran magnitud. Se evaluó el comportamiento de la suma utilizando valores de b menores o iguales que 1. Los resultados obtenidos se presentan en la Figura 3.

También se repitió el experimento utilizando valores negativos de b con tipo entero. Los resultados se muestran en la Figura 4.

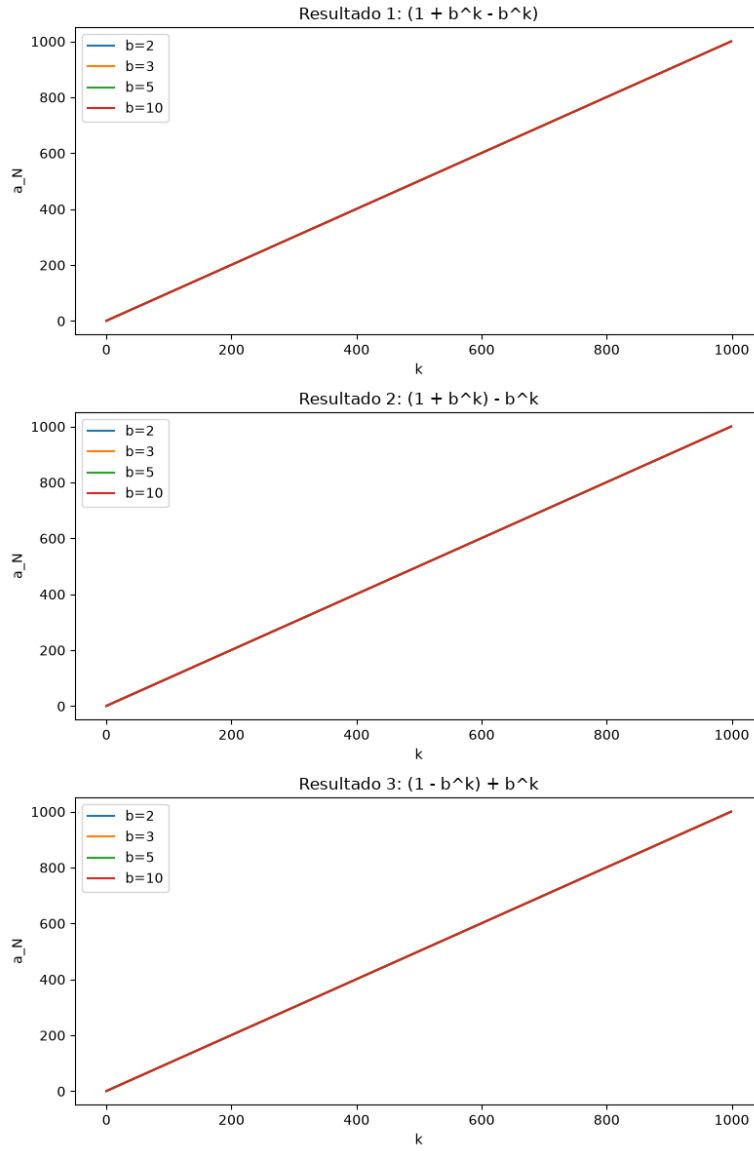


Figura 1: Resultados de las tres expresiones utilizando números enteros, con $b \in \{2, 3, 5, 10\}$ y $N = 1,000$.

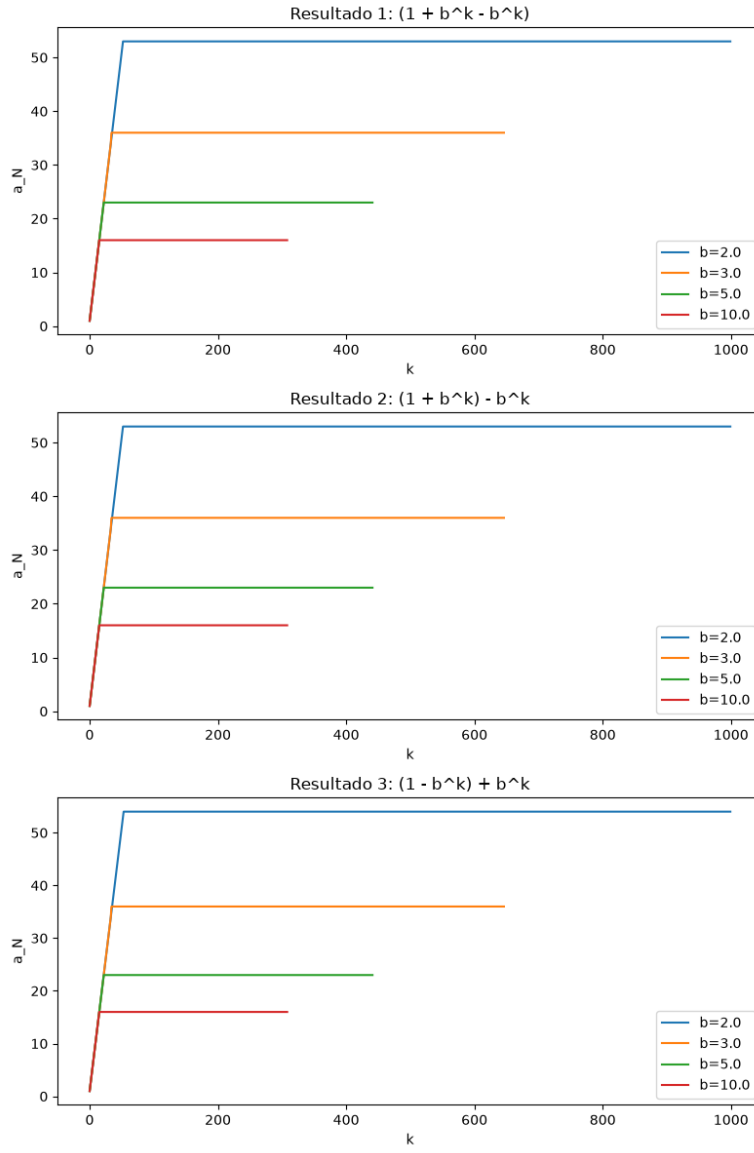


Figura 2: Resultados de las tres expresiones utilizando números de punto flotante, con $b \in \{2,0, 3,0, 5,0, 10,0\}$ y $N = 1,000$.

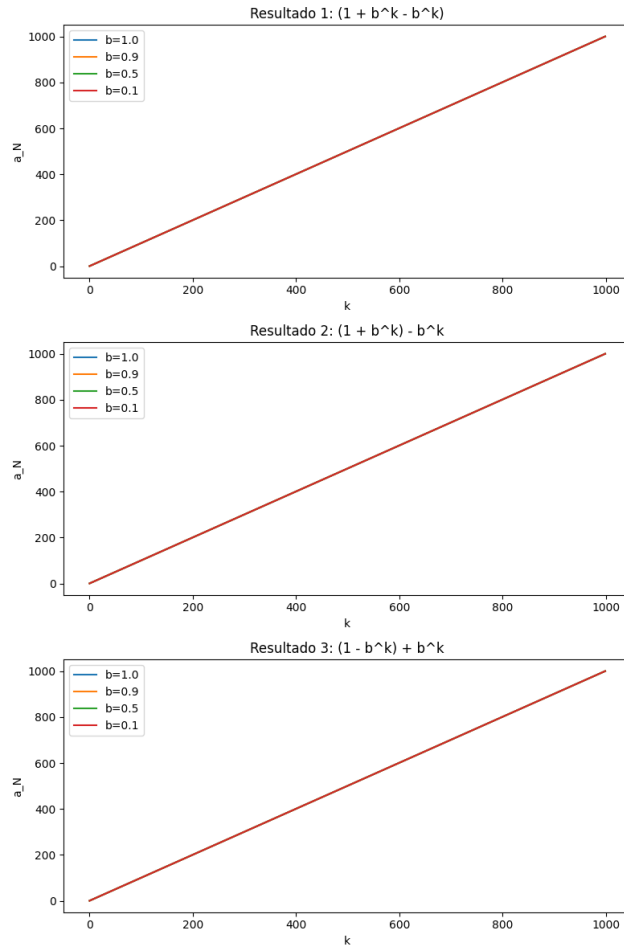


Figura 3: Resultados de las tres expresiones utilizando números de punto flotante b menores o iguales que 1 con $b \in \{1,0, 0,9, 0,5, 0,1\}$ y $N = 1,000$.

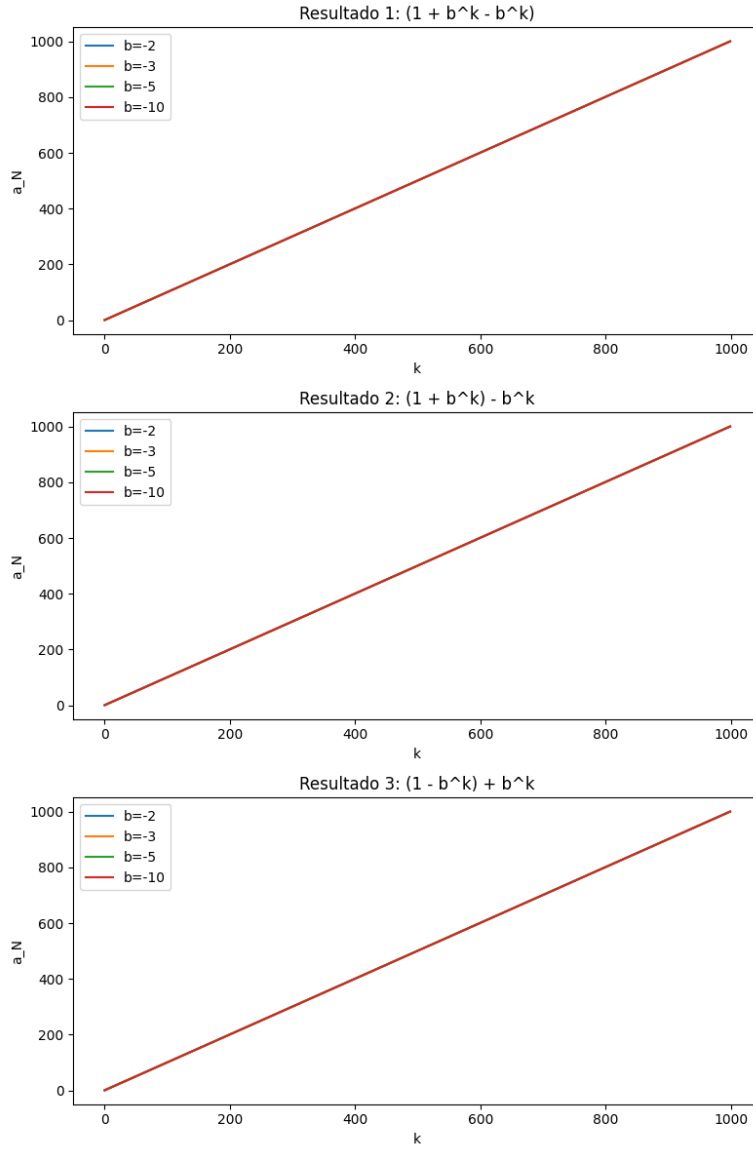


Figura 4: Resultados de las tres expresiones utilizando números enteros, con $b \in \{-2, -3, -5, -10\}$ y $N = 1,000$.

Finalmente, se analizó el comportamiento utilizando valores negativos de b de tipo punto flotante. Los resultados obtenidos se presentan en la Figura 5.

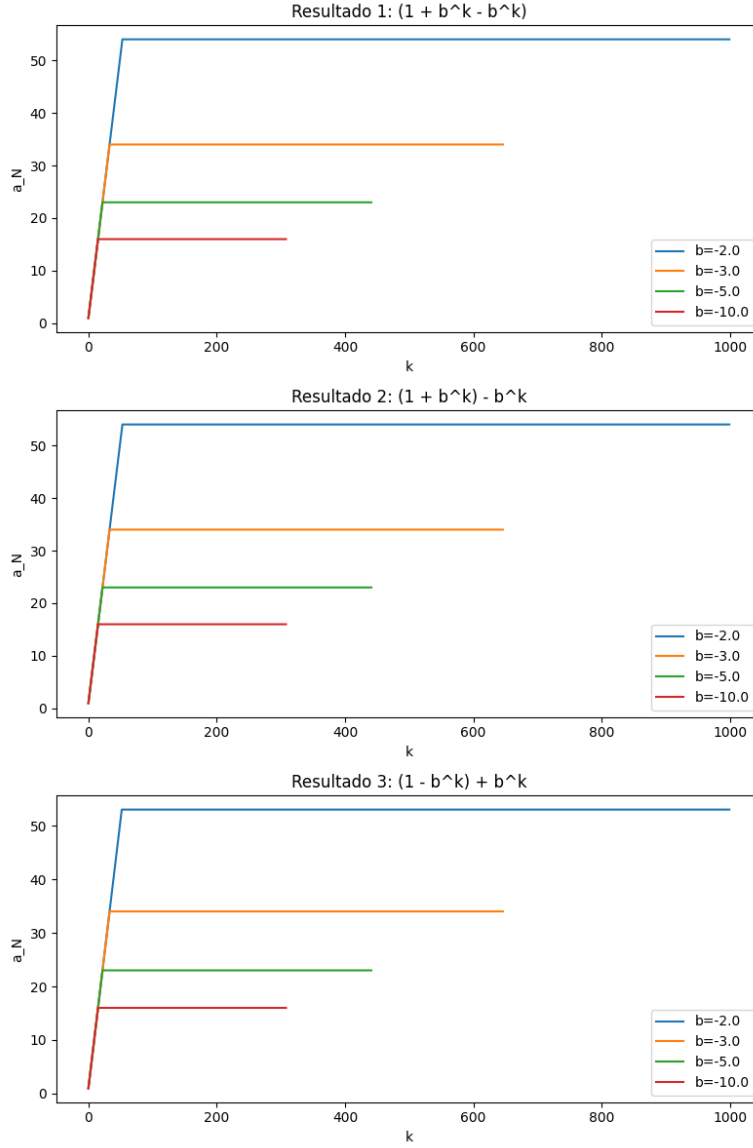


Figura 5: Resultados de las tres expresiones utilizando números de punto flotante, con $b \in \{-2,0, -3,0, -5,0, -10,0\}$ y $N = 1,000$.

4.2. Propiedad conmutativa

En la Figura 6 se observa la evolución del error relativo para los cuatro algoritmos en un rango de hasta 10,000 términos.

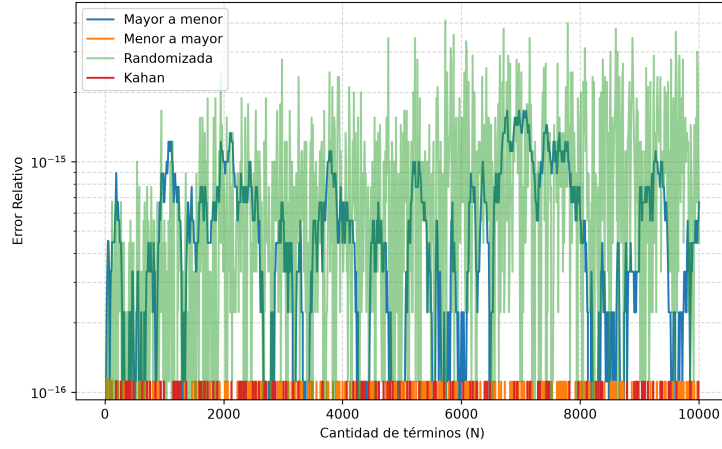


Figura 6: Error relativo de los distintos algoritmos de suma para $N \in [10, 10,000]$. Se utiliza escala logarítmica en el eje de las ordenadas.

Al extender el rango de evaluación hasta 1,000,000 de términos, el comportamiento asintótico del error de la suma secuencial tradicional se vuelve más evidente, tal como se muestra en la Figura 7.

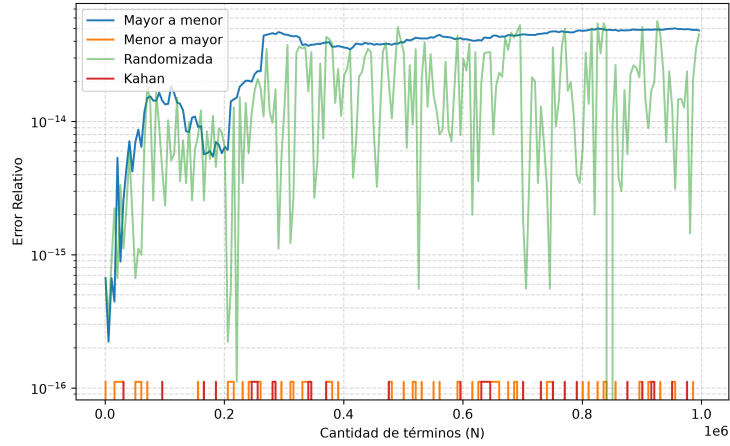


Figura 7: Error relativo de los algoritmos de suma para $N \in [1,000, 1,000,000]$ (escala logarítmica).

Finalmente, la Tabla 1 resume las fluctuaciones del error relativo al aplicar la suma randomizada sobre el mismo conjunto de datos repetidas veces.

Tabla 1: Estadísticas del error relativo para la suma randomizada ($N = 500,000$, 50 ejecuciones)

Métrica	Valor
Error Mínimo	2,2205e-16
Error Máximo	5,5733e-14
Media del Error	2,4507e-14
Desviación Estándar	1,4836e-14

4.3. Representaciones de b_N

En primer lugar, se compararon las dos formas de calcular la sucesión b_N . La Figura 8 muestra el error relativo de ambas representaciones respecto de la fórmula cerrada $\frac{N}{N+1}$ para valores de N entre 1 y 10,000.

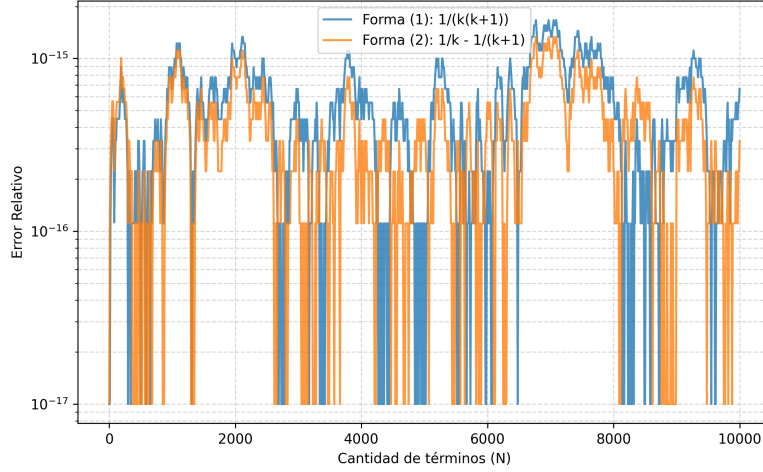


Figura 8: Error relativo de las dos representaciones de b_N para $1 \leq N \leq 10,000$. Se utiliza una escala logarítmica en el eje vertical.

Se observa que ambas representaciones presentan errores relativos muy pequeños durante todo el intervalo analizado. Las dos curvas muestran un comportamiento similar, aunque presentan fluctuaciones y diferencias en distintos valores de N . En ningún caso se observa un crecimiento sostenido del error a medida que aumenta la cantidad de términos.

4.4. Comparación de las formas cerradas

Como parte del primer bonus, se compararon las expresiones

$$1 - \frac{1}{N+1}$$

y

$$\frac{N}{N+1}.$$

La Figura 9 presenta el error relativo de ambas expresiones respecto del valor exacto $\frac{N}{N+1}$ para $10 \leq N \leq 10,000$.

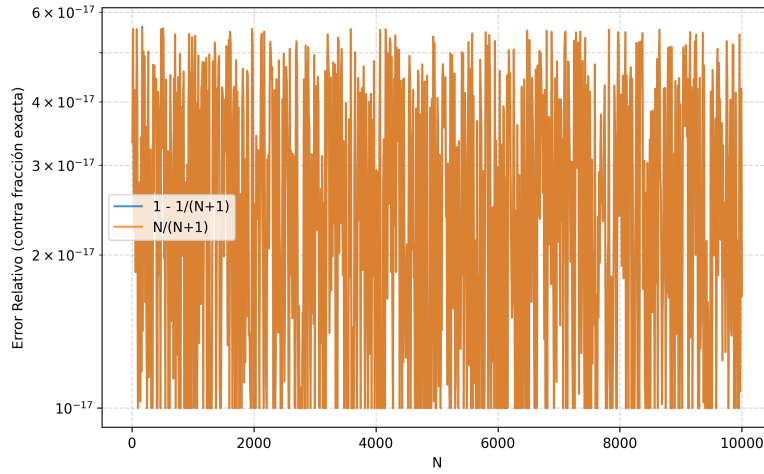


Figura 9: Error relativo de las dos formas cerradas de b_N respecto del valor exacto, para $10 \leq N \leq 10,000$.

Los resultados muestran que ambas expresiones presentan errores relativos muy pequeños y de magnitud similar. La diferencia entre las dos formas también fue evaluada directamente para valores de N entre 1 y 1,000,000, encontrándose 23 casos en los que los resultados numéricos son diferentes.

4.5. Representaciones de c_N

La Figura 10 muestra el error relativo de las dos representaciones de c_N respecto del valor de referencia calculado con 50 dígitos, para $1 \leq N \leq 10,000$.

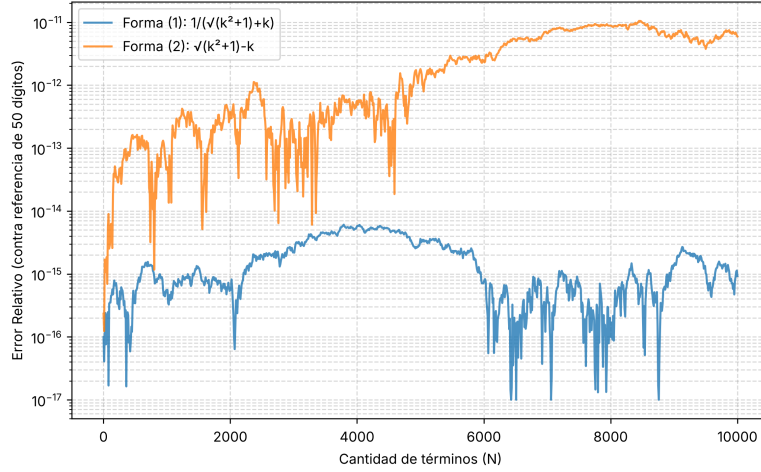


Figura 10: Error relativo de las dos representaciones de c_N respecto de la referencia de 50 dígitos, para $1 \leq N \leq 10,000$ (escala logarítmica).

A diferencia de lo observado para b_N , en este caso las dos representaciones se comportan de forma muy distinta. La forma racionalizada mantiene errores del orden de 10^{-15} en todo el intervalo, mientras que la resta de raíces presenta errores entre tres y cuatro órdenes de magnitud mayores. La Tabla 2 resume estos valores.

Tabla 2: Error relativo de las representaciones de c_N para $1 \leq N \leq 10,000$.

Representación	Máximo	Media	En $N = 10,000$
$c_N^{(1)}: 1/(\sqrt{k^2+1}+k)$	6,07e-15	1,73e-15	9,26e-16
$c_N^{(2)}: \sqrt{k^2+1}-k$	1,05e-11	3,08e-12	5,95e-12

La Figura 11 presenta la diferencia absoluta D_N entre ambas representaciones. Esta diferencia crece desde valores del orden de 10^{-16} para $N = 10$ hasta aproximadamente $1,8 \cdot 10^{-13}$ para $N = 1,000$ y $2,8 \cdot 10^{-11}$ para $N = 10,000$. Por ejemplo, para $N = 10,000$ se obtuvo $c_N^{(1)} = 4,784787737051912$ y $c_N^{(2)} = 4,784787737080387$, mientras que el valor de referencia es $4,784787737051916$.

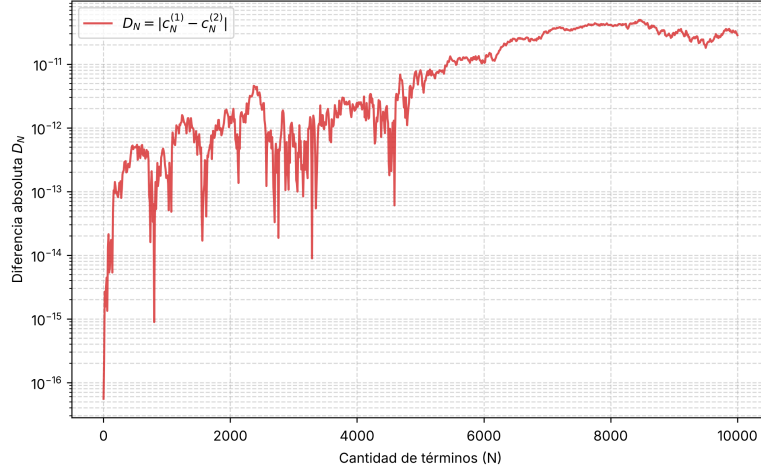


Figura 11: Diferencia absoluta $D_N = |c_N^{(1)} - c_N^{(2)}|$ entre las dos representaciones de c_N (escala logarítmica).

4.6. Términos individuales de c_N

Como parte del segundo bonus, la Figura 12 muestra el error relativo de cada forma de calcular el término k -ésimo de c_N , para $1 \leq k \leq 10^{10}$. La línea punteada corresponde a la estimación teórica εk^2 , con $\varepsilon = 2^{-52} \approx 2,2 \cdot 10^{-16}$, que se justifica en la Discusión.

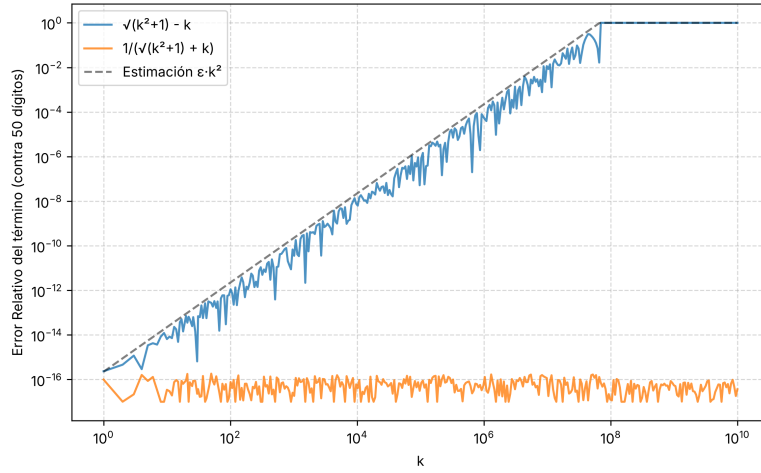


Figura 12: Error relativo de los términos $\sqrt{k^2 + 1} - k$ y $1/(\sqrt{k^2 + 1} + k)$ en función de k (escala logarítmica en ambos ejes).

El error de la forma racionalizada se mantiene en torno a 10^{-16} para todos los valores de k , mientras que el de la resta crece de forma sostenida. La Tabla 3 indica el primer valor de k de la grilla en el que el error relativo de la resta supera cada umbral. A partir de $k \approx 7 \cdot 10^7$ la resta devuelve exactamente 0, lo que corresponde a un error relativo del 100 %.

Tabla 3: Primer valor de k en el que el error relativo de $\sqrt{k^2 + 1} - k$ supera cada umbral.

Umbral de error relativo	Primer k
10^{-14}	9
10^{-12}	101
10^{-10}	1,077
10^{-8}	9,118
10^{-6}	72,804
10^{-4}	1,161,884
10^{-2}	8,757,010
Resultado igual a 0	69,921,627

5. Discusión

5.1. Análisis de la asociatividad

Los resultados muestran una diferencia clara entre usar `int` y `float`. Con enteros (Figura 1) las tres asociaciones dan $a_N = N$ para todos los valores de b y todo N , es decir, el resultado coincide con el valor teórico. Con números de punto flotante (Figura 2) a_N crece linealmente al principio pero luego se estanca en un valor constante que depende de b .

¿Por qué ocurre con float?. Un `float` de doble precisión guarda la parte significativa del número con 53 bits, por lo que solo puede distinguir números cuya diferencia relativa sea de aproximadamente 2^{-53} . Cuando b^k es muy grande el número 1 queda por debajo de la resolución disponible a esa magnitud y la suma $1 + b^k$ se redondea a b^k . Este fenómeno se conoce como *absorción*. En ese caso, el término $(1 + b^k) - b^k$ vale 0 en lugar de 1, y el acumulado deja de aumentar. Esto ocurre cuando b^k supera aproximadamente $2^{53} \approx 9,0 \cdot 10^{15}$, o sea para $k \approx 53/\log_2(b)$, lo que da valores del orden de 53, 33, 23 y 16 para $b = 2, 3, 5$ y 10. Los valores observados (53, 36, 23 y 16) son consistentes con esta estimación y la pequeña diferencia para $b = 3$ se debe a que 3^k tampoco se representa exactamente como el umbral exacto depende de cómo se redondea cada potencia.

Con `int` esto no sucede porque Python utiliza enteros de precisión arbitraria: b^k se calcula de forma exacta, sin redondeo, y la cancelación $b^k - b^k$ es exacta.

Estancamiento y *overflow*. En la Figura 2 se ve además que las curvas de $b = 3,0$, $5,0$ y $10,0$ terminan antes de $k = 1,000$. Esto es un fenómeno distinto de la absorción, el mayor `float` representable es aproximadamente $1,8 \cdot 10^{308}$, y al superarlo Python lanza un `OverflowError`. Esto ocurre en $k = 647$ para $b = 3,0$, en $k = 442$ para $b = 5,0$ y en $k = 309$ para $b = 10,0$, y es la razón por la que se capturó la excepción y se cortó la serie. Para $b = 2,0$ no hay *overflow* porque $2^{1,000} \approx 10^{301}$ todavía es representable.

Efecto de la asociación. Las tres asociaciones producen resultados casi idénticos aunque no exactamente. Para $b = 2,0$ las dos primeras se estancan en 53 y la tercera en 54. Esto se debe a que $1 - 2^{53} = -(2^{53} - 1)$ sí es representable de forma exacta, por lo que en $(1 - b^k) + b^k$ la absorción ocurre un paso después. Esto confirma que cuando utilizamos números de punto flotante la forma de agrupar las operaciones puede influir en el resultado aunque en este caso el efecto sea pequeño en otro contexto puede ser inadmisibile.

¿Se puede observar este fenómeno con $b \leq 1$? No. Si $0 < b < 1$, b^k tiende a 0 y el término que podría ser absorbido es b^k y no el 1. En $(1 + b^k) - b^k$, la suma $1 + b^k$ se redondea con un error de a lo sumo del orden de 10^{-16} , que no se acumula de forma apreciable, al probar con $b = 0,5$ y $b = 0,1$ se obtuvo $a_N = 1,000$ para $N = 1,000$. Para $b = 1$ y $b = 0$ las operaciones son exactas. Es decir, el estancamiento requiere que $|b^k|$ sea muy grande respecto de 1.

¿Pueden tener problemas los enteros con números grandes? Sí, en general. En la mayoría de los lenguajes (por ejemplo C o Java) los enteros tienen un ancho fijo (32 o 64 bits), por lo que existe un valor máximo ($2^{31} - 1$ o $2^{63} - 1$). Al superarlo ocurre un *overflow* y el valor “da la vuelta” (*wrap-around*), produciendo un resultado incorrecto, y normalmente sin ningún aviso. Por ejemplo, con $b = 10$ y enteros de 64 bits, b^k ya se desborda para $k = 19$.

En Python esto no ocurre, los `int` tienen precisión arbitraria y crecen mientras haya memoria disponible y es por eso que los resultados de la Figura 1 son exactos. La contrapartida es que las operaciones se vuelven más lentas a medida que los números crecen.

¿Qué ocurre con b negativo? Con `int` el resultado es exacto: $a_N = N$. Con `float` las curvas también se estancan, y el *overflow* ocurre en los mismos k que con b positivo, ya que solo depende de $|b^k|$. Lo que cambia con el signo es cuál de las asociaciones se estanca un paso después: para $b = -2,0$ las dos primeras se estancan en 54 y la tercera en 53, el orden inverso al de $b = 2,0$. Esto ocurre porque para $k = 53$ (impar) se cumple que $1 + b^k = 1 - 2^{53}$ es representable, mientras que $1 - b^k = 1 + 2^{53}$ no lo es y se absorbe. Para $b = -3,0$, $-5,0$ y $-10,0$ los valores de estancamiento son 36,23 y 16.

5.2. Análisis del orden de los términos

La comparación de los cuatro algoritmos de suma revela comportamientos drásticamente distintos frente al error de truncamiento. Como se observa en la Tabla 1 y en las Figuras 6 y 7, el método clásico de sumar de mayor a menor módulo es el más ineficiente. Al procesar primero los valores grandes, el acumulador principal crece, sometiendo a los términos diminutos subsiguientes al fenómeno de *absorción* (detallado en la Sección 5.1). Al perderse en el límite de los 53 bits de la mantisa, estos decimales se truncan, provocando que el error aumente con N hasta alcanzar magnitudes del orden de 10^{-14} , y luego se mantenga en ese nivel.

En contraposición, sumar de menor a mayor resultó sumamente efectivo, logrando una precisión casi idéntica a la del algoritmo de Kahan. Al comenzar por los valores mínimos, estos logran acumularse con alta fidelidad antes de integrarse a los términos de mayor magnitud (evitando la absorción temprana). El algoritmo de Kahan demostró ser el más robusto, manteniendo el error al límite de la precisión de la máquina (10^{-16}) independientemente del tamaño de N , gracias a su variable de compensación.

Por su parte, la suma randomizada (Tabla 1) osciló entre los peores y mejores casos dependiendo de la permutación, aportando la comprobación empírica de que la suma computacional carece de asociatividad estricta: alterar aleatoriamente el orden de los mismos 500.000 sumandos arrojó una desviación estándar de $1,48 \times 10^{-14}$ entre ejecuciones.

5.3. Análisis de las representaciones de b_N

Las dos formas utilizadas para calcular b_N son matemáticamente equivalentes y, como se observa en la Figura 8, ambas producen resultados con errores relativos muy pequeños. Sin embargo, los errores no son exactamente iguales y aparecen fluctuaciones a medida que aumenta N .

La primera representación calcula directamente cada término mediante

$$\frac{1}{k(k+1)},$$

mientras que la segunda obtiene cada término mediante la resta

$$\frac{1}{k} - \frac{1}{k+1}.$$

Aunque ambas expresiones son equivalentes en aritmética exacta, la segunda requiere realizar una resta entre números muy próximos, especialmente para valores grandes de k . Esto puede causar una pérdida de precisión debido a la cancelación de cifras significativas. Por este motivo, pequeñas diferencias entre ambas representaciones pueden aparecer al realizar los cálculos con números de punto flotante.

A pesar de estas diferencias, los errores observados en el rango analizado se mantienen en valores muy pequeños. Por lo tanto, para esta sucesión y para los

valores de N utilizados, ambas representaciones presentan un comportamiento numéricamente similar.

5.4. Análisis de las formas cerradas

En el caso de las expresiones

$$1 - \frac{1}{N+1}$$

y

$$\frac{N}{N+1},$$

ambas representan exactamente el mismo valor. Sin embargo, las computadoras al utilizar la aritmética de punto flotante, causan pequeñas diferencias de redondeo.

En la Figura 9 se observa que los errores relativos de ambas expresiones son extremadamente pequeños y de magnitudes similares. Esto indica que, dentro del rango analizado, ninguna de las dos representaciones presenta una pérdida significativa de precisión.

Sin embargo, al comparar directamente los resultados para valores de N entre 1 y 1 000 000, se encontraron algunos casos en los que las expresiones producen valores de punto flotante diferentes. Esto demuestra que la equivalencia matemática de dos expresiones no garantiza que sus representaciones computacionales sean idénticas.

5.5. Análisis de las representaciones de c_N

Las dos representaciones de c_N no producen exactamente los mismos resultados: D_N es distinto de cero desde los primeros valores de N y crece a medida que se agregan términos. Al comparar cada forma con la referencia de alta precisión (Tabla 2) se observa que prácticamente toda la diferencia proviene de la resta de raíces, cuyo error relativo es del orden de 10^{-11} , mientras que la forma racionalizada mantiene errores del orden de la precisión de máquina. Es decir, $D_N \approx c_N \cdot E_{\text{rel}}(c_N^{(2)})$.

La causa de esta diferencia es la *cancelación catastrófica*. Para valores grandes de k se cumple que

$$\sqrt{k^2 + 1} \approx k + \frac{1}{2k},$$

por lo que en $\sqrt{k^2 + 1} - k$ se restan dos números casi iguales. La raíz calculada en punto flotante tiene un error de redondeo de hasta medio *ulp*, es decir, un error absoluto del orden de $\varepsilon k/2$. La resta en sí es exacta, pero el resultado es muy pequeño, del orden de $1/(2k)$, y los dígitos comunes de ambos operandos se cancelan, dejando como dígitos significativos justamente aquellos afectados por el redondeo. En consecuencia, el error relativo del término es aproximadamente

$$\frac{\varepsilon k/2}{1/(2k)} = \varepsilon k^2,$$

lo que coincide con la tendencia observada en la Figura 12. Como plantea Goldberg (1991), la resta no genera el error, sino que pone en evidencia el error introducido previamente por el cálculo de la raíz.

En el caso extremo, cuando $\frac{1}{2k}$ es menor que la mitad de la separación entre dos `float` consecutivos cercanos a k (lo que ocurre para $k > 2^{26} \approx 6,7 \cdot 10^7$), la raíz $\sqrt{k^2 + 1}$ se redondea exactamente a k y el término vale 0. Esto es consistente con el valor observado en la Tabla 3.

La forma racionalizada evita este problema porque reemplaza la resta por una suma de dos números positivos seguida de una división. Ninguna de estas operaciones amplifica los errores relativos, por lo que cada término conserva un error del orden de ε para cualquier valor de k .

Respecto de a partir de qué valor de k la pérdida de precisión es apreciable, la respuesta depende del criterio elegido. Ya con $k \approx 100$ el error (10^{-12}) es miles de veces mayor que la precisión de máquina; con $k \approx 10^4$ se pierden aproximadamente la mitad de los 16 dígitos significativos (10^{-8}); con $k \approx 10^7$ quedan solo dos o tres dígitos correctos, y a partir de $k \approx 7 \cdot 10^7$ no queda ninguno. En la sumatoria c_N con $N \leq 10,000$ el efecto total se mantiene acotado (10^{-11}) porque los términos con k grande son pequeños, pero para valores de N mayores a 2^{26} la resta de raíces dejaría de crecer por completo, a pesar de que el valor real de c_N sigue aumentando.

5.6. Comparación de errores / comportamiento numérico

Los tres experimentos muestran tres mecanismos distintos por los cuales el resultado computacional se aleja del valor teórico, con consecuencias de magnitud muy diferente:

- **Absorción:** al sumar un número pequeño a otro mucho mayor, el pequeño se pierde. En la propiedad asociativa con `float` este efecto es extremo: $a_{1,000}$ vale 53 en lugar de 1,000 para $b = 2,0$, es decir, un error de más del 90 %. En el análisis del orden de los términos el mismo fenómeno, aplicado de forma gradual, explica por qué sumar de mayor a menor módulo produce los peores resultados.
- **Acumulación de errores de redondeo:** cada operación aporta un error del orden de ε , y al repetirla miles o millones de veces el error crece con N . Este efecto es el que reducen la suma de menor a mayor módulo y, sobre todo, la suma de Kahan, que mantiene el error en el orden de 10^{-16} incluso para $N = 10^6$.
- **Cancelación catastrófica:** al restar números casi iguales se amplifica el error relativo de los operandos. Es la causa de que la resta de raíces en c_N tenga un error entre tres y cuatro ordenes de magnitud mayor que la forma racionalizada, y de que cada término llegue a perder toda su precisión.

En contraste, las representaciones de b_N y sus dos formas cerradas presentaron errores del orden de la precisión de máquina en todos los casos. Esto muestra que no toda reescritura de una expresión es problemática: lo determinante es cuánto error absoluto aporta cada término al total. Una resta de cantidades próximas es problemática cuando sus operandos son grandes (como en c_N), pero poco problemática cuando son pequeños y decrecientes (como en b_N).

6. Conclusiones

Los experimentos realizados confirman que propiedades básicas de la aritmética exacta no se conservan necesariamente cuando las operaciones se realizan en una computadora con números de punto flotante.

En primer lugar, la propiedad asociativa deja de cumplirse con `float`: la suma a_N , que en teoría vale N , se estanca en cuanto b^k supera aproximadamente 2^{53} , debido a la absorción del término 1. Con los enteros de Python, en cambio, el resultado es exacto gracias a su precisión arbitraria, aunque en lenguajes con enteros de tamaño fijo el riesgo pasa a ser el *overflow*. La forma de agrupar las operaciones también influyó en el resultado, aunque en este caso de manera leve.

En segundo lugar, el orden en que se suman los términos tiene un efecto medible sobre el error. Sumar de mayor a menor módulo fue la peor estrategia, mientras que sumar de menor a mayor módulo y, especialmente, la suma compensada de Kahan permitieron mantener el error relativo cerca de la precisión de máquina incluso con un millón de términos. La suma randomizada dio resultados distintos en cada ejecución, lo que evidencia que la suma en punto flotante no es asociativa.

En tercer lugar, dos expresiones matemáticamente equivalentes pueden tener comportamientos numéricos muy distintos. Para b_N las diferencias entre representaciones fueron del orden de la precisión de máquina, pero para c_N la resta de raíces resultó afectada por cancelación catastrófica, con un error relativo del orden de εk^2 en cada término, llegando a perder todos los dígitos significativos para $k > 2^{26}$. Reescribir la expresión en su forma racionalizada eliminó por completo este problema.

En síntesis, el error numérico no se debe a un defecto del lenguaje de programación, sino a la representación finita de los números, y puede controlarse con decisiones adecuadas: elegir el tipo de dato apropiado, ordenar los términos o usar algoritmos de suma compensada, y evitar restas entre cantidades muy próximas reescribiendo las expresiones. Como posibles extensiones del trabajo se podrían estudiar otros algoritmos, como la suma por pares (*pairwise summation*) o la función `math.fsum` de Python, y repetir los experimentos en precisión simple para observar cómo cambian los umbrales encontrados.

7. Bibliografía

Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1), 5–48.
<https://doi.org/10.1145/103162.103163>

Higham, N. J. (2002). *Accuracy and stability of numerical algorithms* (2.^a ed.). Society for Industrial and Applied Mathematics.
<https://doi.org/10.1137/1.9780898718027>

IEEE Computer Society. (2019). *IEEE standard for floating-point arithmetic* (IEEE Std 754-2019). IEEE.
<https://doi.org/10.1109/IEEESTD.2019.8766229>

Kahan, W. (1965). Pracniques: Further remarks on reducing truncation errors. *Communications of the ACM*, 8(1), 40.
<https://doi.org/10.1145/363707.363723>

OpenStax. (2022). Series y sus notaciones. En *Precálculo* (2.^a ed., sección 11.4). <https://openstax.org/books/prec%C3%A1lculo-2ed/pages/11-4-series-y-sus-notaciones>

Anthropic. (2026). Claude (Sonnet 5.5) [Modelo de lenguaje grande]. (2026).
<https://claude.ai/>

Python Software Foundation. (s.f.-a). *decimal: Decimal fixed-point and floating-point arithmetic*. Python 3 documentation.
<https://docs.python.org/3/library/decimal.html>

Python Software Foundation. (s.f.-b). *Floating-point arithmetic: Issues and limitations*. Python 3 documentation.
<https://docs.python.org/3/tutorial/floatingpoint.html>

Universidad Católica del Uruguay. (2026). *Cálculo aplicado: Optimización numérica en una dimensión* [Letra del trabajo práctico]. Facultad de Ingeniería y Tecnología, Departamento de Ciencias Exactas y Naturales.